Quantum Information Report

Francesca Pagano

Shor's Algorithm

Shor's algorithm is a quantum algorithm for integer factorization. This problem has many implications, one of the most famous one is in cryptography, in particular for the RSA scheme, used in online transactions. The fastest classical algorithm for factorization runs in sub-exponential time $e^{\mathcal{O}(n^{1/3}(\log n)^{2/3})}$. Instead, Shor's quantum algorithm achieves polynomial time $\mathcal{O}(n^3)$, where $n = \lceil \log_2 N \rceil$ is the number of bits of N.
The speedup is obtained thanks to quantum computations based on *quantum phase estimation*. The algorithm exists also thanks to classical results that enable the reduction of integer factorization to phase estimation and the subsequent extraction of the factors from the measured phase.

1. Classical Reduction

If N is the odd composite number we want to factorize, one should then pick an integer a such that $1 < a < N$ and $\gcd(a, N) = 1$. We define **order** of a modulo N the smallest positive integer r such that $a^r \equiv 1 \pmod N$, or, equivalently, we can define r as the period of the function $f(x) = a^x \bmod N$. If r is even and $a^{r/2} \not\equiv -1 \pmod N$, then from $a^r \equiv 1 \pmod N$ one obtains

$$\Big(a^{r/2} - 1\Big)\Big(a^{r/2} + 1\Big) \equiv 0 \pmod N, \tag{1}$$

and the prime factors can be recovered via $\gcd(a^{r/2} \pm 1,\, N)$.
Another classical aspect concerns the quantum order-finding subroutine measurement: we need to convert the measured phase φ into the order r. If $\varphi = m/2^{n_x}$, one computes the continued fraction expansion of φ and tests candidate denominators q until $a^q \equiv 1 \pmod N$.

2. Quantum Phase Estimation

QPE is a quantum algorithm that allows the estimation of the eigenphases of a unitary operator. Shor's algorithm is an instance of QPE where the unitary operator is:

$$U_a \left|x\right\rangle = \begin{cases} \left|ax \bmod N\right\rangle & \text{if } 0 \le x < N, \\ \left|x\right\rangle & \text{if } N \le x < 2^n. \end{cases} \tag{2}$$

The eigenstates of U_a are the states $\left|u_s\right\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} \left|a^k \bmod N\right\rangle$, where $s = 0, \ldots, r-1$, with corresponding eigenphase $\phi_s = s/r$. The target register is initialised to $\left|1\right\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \left|u_s\right\rangle$, making all eigenstates available without explicitly coding the unavaible period. The controlled-$U_a^{2^k}$ operations imprint $e^{2\pi i \cdot 2^k \phi_s}$ in the ancilla qubit k via phase kickback, producing the state $\frac{1}{2^{n/2}} \sum_{j=0}^{2^n-1} e^{2\pi i \phi_s j} \left|j\right\rangle$. The inverse Quantum Fourier Transform will provide an estimate of ϕ_s since the probability will have a peak near $2^n \phi_s$.

Quantum Fourier Transform

On an n-qubit register, the QFT maps computational basis states in the fourier basis as

$$\mathrm{QFT}\left|x\right\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i x y/2^n} \left|y\right\rangle, \qquad x \in \{0, \ldots, 2^n - 1\}. \tag{3}$$

Writing x in binary as $x = x_{n-1}2^{n-1} + \cdots + x_0$, the QFT factorises into a tensor product of single-qubit states:

$$\mathrm{QFT}\left|x\right\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{k=0}^{n-1} \Big(\left|0\right\rangle + e^{2\pi i\, 0.x_k \cdots x_0} \left|1\right\rangle\Big), \tag{4}$$

where $0.x_k \cdots x_0 = x_k/2 + x_{k-1}/4 + \cdots$ is the binary fraction. This product reflects the structure of the circuit:

1. **Hadamard gate:** H is applied to qubit i, producing $(|0\rangle + e^{2\pi i\, 0.x_i}\,|1\rangle)/\sqrt{2}$.
2. **Controlled phase rotations:** for each $j < i$, a CR_k gate with $k = i - j$ is applied, where $R_k = \mathrm{diag}(1,\, e^{i\pi/2^k})$. These accumulate the remaining bits $0.x_i x_{i-1} \cdots x_0$ on qubit i. In the code: `qc.cp(pi / 2**(e+1), j, i)`, with `e` $= i - j - 1$.
3. **Bit-reversal SWAPs:** the natural output has qubits in reversed order. A final layer of SWAP gates corrects this: `qc.swap(i, n-i-1)` for $i = 0, \ldots, \lfloor n/2 \rfloor - 1$.

The inverse QFT is obtained by reversing the gate sequence and conjugating all phases. In the python notebook it is called via `myqft(nx).inverse()` and appended to the control register at the end of QPE.

Modular exponentiation unitary

A good implementation of U_a is relevant for the performance of the algorithm, as it dominates the gate count and depth. Modular exponentiation is implemented via a sequence of modular multiplications, which relies on modular addition, which relies on ripple-carry addition.
This implementation follows the VBE construction, chosen because it is not hard-coded in N and offers a good balance between simplicity of the implementation and efficiency.ì In the simulations it was efficient enough to support factorization up to 8-bit integers (limited by number of qubits of the simulator).

Ripple-Carry Adder One method that allows to perform additions with classical logic gates is the ripple-carry adder. It computes the sum in the saem way one would do it by hand: $\mathrm{sum}_i = a_i \oplus b_i \oplus c_{i-1}$, $c_i = a_i b_i \oplus c_{i-1}(a_i \oplus b_i)$. The two elementary gates used in the quantum version are the *CNOT*, which implements $|a, b\rangle \to |a, a \oplus b\rangle$, quantum analog of XOR, and the *Toffoli*, which implements $|a, b, c\rangle \to |a, b, c \oplus ab\rangle$, analog of AND followed by XOR.
The *CARRY gate* computes c_i using three steps: **(1)** CCX(a_i, b_i, c_i): adds $a_i b_i$ into c_i, **(2)** CX(a_i, b_i): overwrites $b_i \leftarrow a_i \oplus b_i$, **(3)** CCX$(c_{i-1}, a_i \oplus b_i, c_i)$: adds $c_{i-1}(a_i \oplus b_i)$ into c_i. After step 3, c_i holds the correct carry. Since a_i is still available, the gate is reversible.
The *SUM gate* implements sum_i as a chain of XORs, implemented by two CNOTs, CX(a_i, b_i) then CX(c_{i-1}, b_i), writing the result into b_i.
The adder considers an initial pass that applies CARRY gates from $i = 0$ to $n - 1$, propagating c_i at each step exactly as a classical ripple-carry adder would. The overflow is stored in $b[n]$. Then CARRY^{-1} is applied in reverse order: this simultaneously computes the sum bits via SUM gates and uncomputes every carry ancilla back to $|0\rangle$.

Modular Adder The modular adder implements $|a, b\rangle \to |a,\, (a + b) \bmod N\rangle$. The key observation is that $a + b \bmod N$ equals $a + b$ if no overflow occurred, and $a + b - N$ otherwise. So, it suffices to detect which case applies and correct accordingly.
The circuit proceeds in five steps:
(1) Add. Compute $b \leftarrow a + b$ using the ripple-carry adder. The overflow bit $b[n]$ is 1 if $a + b \geq 2^n$, but we care about whether $a + b \geq N$.
(2) Subtract N**.** Swap $a \leftrightarrow b_N$ and apply the inverse adder to compute $b \leftarrow a + b - N$. After this, $b[n] = 0$ signals underflow ($a + b < N$), meaning we subtracted too much.
(3) Detect underflow. Copy the underflow condition into a flag qubit t via X$(b[n])$, CX$(b[n], t)$, X$(b[n])$. The inversions are necessary to restore $b[n]$ without destroying it, since t must be set coherently to remain entangled with the computation.

(4) Correct. If $t = |1\rangle$ (underflow), add N back to b by conditionally reconstructing N in register a bit by bit via $\mathrm{CX}(t, a_i)$ for each set bit of N, then applying the adder. The reconstruction is then uncomputed to restore a.
(5) Uncompute t. Swap $a \leftrightarrow b_N$ back, apply the inverse adder to undo step (1), then reset t by checking $b[n]$ again. A final adder call restores the correct result in b. After all uncomputation, $t = |0\rangle$, all carry ancillas are clean, and b holds $a + b \bmod N$.

Controlled multiplier The gate implements $|x\rangle|z\rangle \to |x\rangle|mz \bmod N\rangle$ by exploiting the binary decomposition $mz = \sum_j z_j \cdot m2^j$. For each bit $z[j]$, the value $m2^j \bmod N$ is conditionally loaded into workspace register a via Toffoli gates gated on both x and $z[j]$, then accumulated into b via the modular adder. Register a is then uncomputed bit by bit with the same Toffoli pattern, restoring $a = |0\rangle$ for the next iteration.

Modular exponentiation Since $a^x = a^{2^0 x_0} \cdot a^{2^1 x_1} \cdots a^{2^{n_x-1} x_{n_x-1}}$, the exponentiation is decomposed into n_x sequential multiplications. At step i, the multiplier is $m_i = a^{2^i} \bmod N$, precomputed classically by repeated squaring. Each step follows:

$$|z,\, 0\rangle \xrightarrow{\text{mult by } m_i} |z,\, m_i z \bmod N\rangle \xrightarrow{\text{c-SWAP}} |m_i z \bmod N,\, z\rangle \xrightarrow{\text{mult by } m_i^{-1}} |m_i z \bmod N,\, 0\rangle\,. \quad (5)$$

The inverse multiplication by $m_i^{-1} \bmod N$ (computed via `mod_inverse`) uncomputes b back to $|0\rangle$, erasing all intermediate partial products from memory.

Registers Some sub-registers are thus required by the VBE modular exponentiation circuit:

- **Target** z (n qubits): initialised to $|1\rangle$; accumulates $a^x \bmod N$ at each step.
- **Workspace** a (n qubits): temporarily encodes the classical multiplier $m \cdot 2^j \bmod N$; uncomputed to $|0\rangle$ after each modular addition.
- **Accumulator** b ($n+1$ qubits): receives the partial product during modular multiplication; the extra qubit handles overflow during modular reduction.
- **Carry** c (n qubits): propagates carry bits in the ripple-carry adder; restored to $|0\rangle$ by uncomputation.
- **Modulus** bN (n qubits): stores N for the conditional subtraction step of the modular adder.
- **Flag** t (1 qubit): records underflow in the modular adder, gating the conditional correction.

Quantum Phase Estimation for Order Finding

The circuit uses two registers: a control register of t qubits and a target register initialised to $|1\rangle$, which decomposes as a uniform superposition over all eigenstates $|u_s\rangle$ of U_a. The QPE proceeds in three stages:

1. **Initialisation.** Hadamard gates prepare the control register in a uniform superposition:

$$|\phi_0\rangle = |0\rangle^{\otimes t}|1\rangle \xrightarrow{H^{\otimes t}} |\phi_1\rangle = \frac{1}{\sqrt{2^t}}\left(|0\rangle + |1\rangle\right)^{\otimes t}|1\rangle. \quad (6)$$

2. **Phase kickback.** The controlled-$U_a^{2^k}$ gates imprint the eigenphase onto each control qubit:

$$|\phi_2\rangle = \frac{1}{\sqrt{2^t}}\left(|0\rangle + e^{\frac{2\pi i s}{r}2^0}|1\rangle\right)\otimes\cdots\otimes\left(|0\rangle + e^{\frac{2\pi i s}{r}2^{t-1}}|1\rangle\right) = \frac{1}{\sqrt{2^t}}\prod_{k=1}^{t}\left(\sum_{\gamma_k=0}^{1} e^{\frac{2\pi i s}{r}\gamma_k 2^{k-1}}|\gamma_k\rangle\right)|1\rangle,$$

which in decimal notation reads

$$|\phi_2\rangle = \frac{1}{\sqrt{2^t}} \sum_{\gamma=0}^{2^t-1} e^{2\pi i \frac{s}{r}\gamma} |\gamma\rangle |1\rangle. \tag{7}$$

3. **Inverse QFT and measurement.** Writing $\phi_s = s/r \approx \alpha/2^t$ with $\alpha \in \{0, \ldots, 2^t - 1\}$, the inverse QFT acts on the control register as:

$$|\phi_3\rangle = \mathrm{QFT}^\dagger |\phi_2\rangle = \frac{1}{2^t} \sum_{\chi=0}^{2^t-1} \sum_{\gamma=0}^{2^t-1} e^{\frac{2\pi i \alpha\gamma}{2^t}} e^{-\frac{2\pi i \chi\gamma}{2^t}} |\chi\rangle |1\rangle. \tag{8}$$

Measurement of the control register yields $\beta \in \{0, \ldots, 2^t - 1\}$ with probability

$$P(\beta) = \left| \frac{1}{2^t} \sum_{\gamma=0}^{2^t-1} e^{2\pi i \gamma \left(\phi_s - \beta/2^t\right)} \right|^2, \tag{9}$$

sharply peaked around $\beta \simeq 2^t \phi_s$. The continued fraction expansion of $\beta/2^t$ then recovers r, verified by checking $a^r \equiv 1 \pmod{N}$.

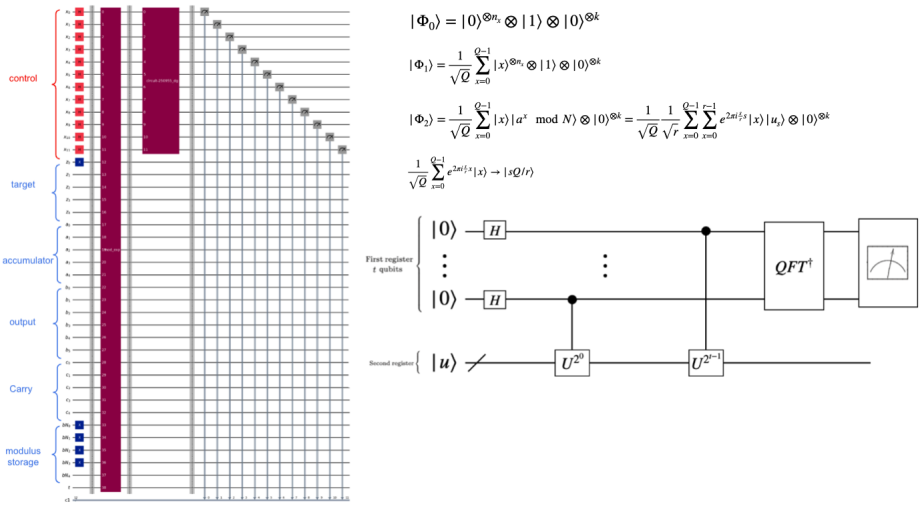

Figure 1: Circuit diagram for Shor's algorithm and generic QPE.

4. Experimental Results

4.1 Measurement Outcomes and Period Recovery

After applying the inverse QFT to the control register, the measurement histogram peaks at multiples of $2^{n_x}/r$. For example, in Fig. 2 there is the case $N = 33$, $a = 7$, the dominant measurement outcome is $m = 29491$ (binary: `111001100110011`), giving $\varphi = 29491/32768 \approx 0.900$. The continued fraction expansion yields the candidate $r = 10$, which satisfies $7^{10} \equiv 1 \pmod{33}$. From this, $\gcd(7^5 - 1, 33) = 3$ and $\gcd(7^5 + 1, 33) = 11$, correctly recovering the factors $3 \times 11 = 33$.

4.2 Scalability

Execution time was measured for $N \in \{15, 55, 65, 133\}$ (corresponding to $L = 4, 6, 7, 8$ bits). On a semi-log scale, the data lie close to the theoretical $\mathcal{O}(L^3)$ reference (Figure 3). The code was running in simulator MPS backend. The observed scaling is consistent with the theoretical polynomial growth of the algorithm's complexity. There are some deviations which can be explained as finite-size effects and overheads of the implementation.

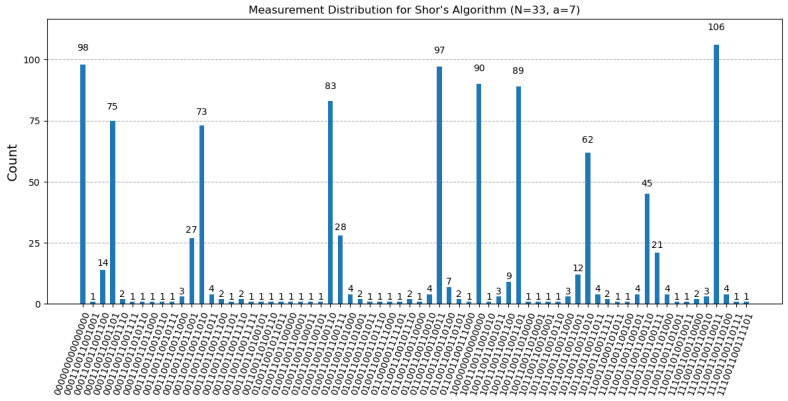

Figure 2: Measurement distribution of the control register after applying the inverse QFT ($N = 33$, $a = 7$).

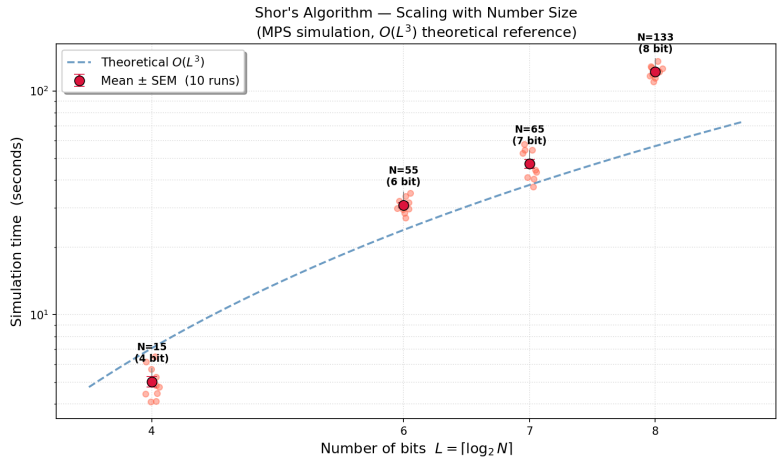

Figure 3: Execution time vs. number of bits $L = \lceil \log_2 N \rceil$ on a semi-log scale. Are included 10 executions to estimate the variability.

5. RSA Application

One application of Shor's algorithm is the breaking of RSA, as it relies on the classical time limitations of factoring the product of two large prime numbers. To encrypt data the receiver chooses two large primes p and q, computes $N = pq$ and $\phi(N) = (p-1)(q-1)$, then picks a public exponent e coprime to $\phi(N)$. Encryption and decryption are defined as $c = m^e \bmod N$ and $m = c^d \bmod N$, with private key $d = e^{-1} \bmod \phi(N)$. Shor's algorithm can be used to find the factors of N. Once p and q are known, $\phi(N)$ can be computed, and the private key d can be derived from the public key e.

References

[1] Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer, Peter W. Shor, 1997.
[2] Quantum Computation and Quantum Information, Michael A. Nielsen and Isaac L. Chuang, 2010.
[3] Quantum Networks for Elementary Arithmetic Operations, V. Vedral, A. Barenco, A. Ekert, 1996.